2.3 Narzędzia Agentowe CLI: Praca z Aider i Open Interpreter – programowanie przez dialog w konsoli

Moduł 2: Ekosystem Vibe Coding i AI-first IDE

Krzysztof Rytlewski · 2026

Narzędzia agentowe CLI – czym są i dlaczego warto je znać?

Narzędzia agentowe CLI (ang. *Command-Line Interface agent tools*) to programy uruchamiane w terminalu, które integrują modele językowe bezpośrednio z powłoką systemową. Działają **niezależnie od edytora** – można je uruchomić na zdalnym serwerze SSH, w kontenerze Docker, w skryptach CI/CD.

Kluczowe cechy:

- **Bez GUI** – interfejs tekstowy, idealne do automatyzacji i środowisk serwerowych.
- **Bezpośredni dostęp do systemu plików** – operują na prawdziwych plikach projektu.
- **Integracja z Git** – narzędzia jak Aider automatycznie commitują zmiany.
- **Model-agnostyczne** – obsługują OpenAI, Anthropic, lokalne modele (Ollama), dając kontrolę nad kosztami i prywatnością.

Aider – programowanie przez dialog z automatycznym commitowaniem

Aider to narzędzie open-source do programowania dialogowego w terminalu. Jego wyróżniającą cechą jest **głęboka integracja z Git** – każda zaakceptowana zmiana jest natychmiast commitowana z automatycznie wygenerowaną wiadomością.

```
# Instalacja
pip install aider-chat

# Uruchomienie z modelem Claude
export ANTHROPIC_API_KEY=sk-ant-...
aider --model claude-3-5-sonnet-20241022

# Wskazanie konkretnych plików kontekstu
aider src/services/user.service.js src/models/user.model.js
```

Podstawowe komendy w sesji Aidra

Komenda	Opis
`/add plik.js`	Dodaje plik do aktywnego kontekstu sesji
`/drop plik.js`	Usuwa plik z kontekstu
`/diff`	Pokazuje bieżące niezacommitowane zmiany
`/undo`	Cofa ostatni commit stworzony przez Aidra
`/run polecenie`	Uruchamia polecenie powłoki i pokazuje wynik modelowi
`/ask pytanie`	Zadaje pytanie bez modyfikowania kodu
`/exit`	Kończy sesję

Automatyczne commitowanie tworzy audytowalną historię pracy AI, co ułatwia przeglądy kodu i rollback.

Aider – zaawansowane tryby pracy

Tryby edycji (`--edit-format`):

- `diff` (domyślny) – model generuje ujednolicone diffy, precyzyjny i oszczędny na tokenach.
- `whole` – model przepisuje cały plik; lepszy przy rozległych zmianach.
- `udiff` – rozszerzony format diff przy skomplikowanych zmianach.

Tryb Architect + Editor – dwa modele równolegle: mocniejszy planuje architekturę, słabszy implementuje szczegóły. Optymalizuje stosunek jakości do kosztu:

```
aider --architect --model o1 --editor-model claude-3-5-haiku-20241022
```

Konfiguracja projektu – plik `.aider.conf.yml`

Plik konfiguracyjny zapisany w katalogu projektu definiuje domyślne zachowanie Aidra dla całego zespołu:

```yaml
model: claude-3-5-sonnet-20241022
auto-commits: true
dirty-commits: false
show-diffs: true
read:
  - README.md
  - docs/architecture.md
```

Tryb `--no-auto-commits` – Aider modyfikuje pliki, ale nie commituje bez zgody użytkownika. Przydatny gdy chcesz samodzielnie kontrolować historię Git.

Open Interpreter – uruchamianie kodu przez model AI

Open Interpreter umożliwia modelowi językowemu **pisanie i wykonywanie kodu** w środowisku użytkownika. Może uruchamiać skrypty, przetwarzać dane, instalować pakiety i operować na systemie plików.

```
# Instalacja i uruchomienie
pip install open-interpreter
interpreter

# Tryb lokalny (bez wysyłania kodu na serwery)
interpreter --local --model ollama/llama3
```

Przykładowe użycie:

```
> Pobierz dane z GitHub API, wyodrębnij 5 ostatnich wersji z datami
  i zapisz do pliku releases.json
```

Open Interpreter – kluczowa różnica w stosunku do Aidra

Aider edytuje pliki projektu i commituje zmiany – skupia się na kodzie źródłowym.

Open Interpreter *wykonuje* kod, a nie tylko go edytuje. Oznacza to, że może:

- bezpowrotnie modyfikować lub usuwać pliki,
- wykonywać żądania sieciowe,
- instalować pakiety w systemie.

Zawsze weryfikuj kod przed akceptacją wykonania. Open Interpreter jest idealny do jednorazowych zadań analitycznych, eksploracji danych i automatyzacji systemowej – nie do edycji bazy kodu.

Bezpieczeństwo – główne ryzyka i mitygacje

Ryzyko	Narzędzie	Mitygacja
Przypadkowe usunięcie plików	Open Interpreter	Używaj w Docker/VM
Wyciek kluczy API przez kontekst	Aider, OI	Dodaj `.env` do `.aiderignore`
Nadpisanie plików produkcyjnych	Aider	Pracuj na osobnej gałęzi Git
Wykonanie złośliwego kodu	Open Interpreter	Czytaj każdy snippet przed akceptacją
Prompt injection przez dane zewnętrzne	OI	Nie przekazuj niezaufanych danych jako kontekstu

```
# Tryb bezpieczny Open Interpreter - pyta przed każdym wykonaniem
interpreter --safe-mode ask
```

Izolowanie pracy w Dockerze

Dla zadań szczególnie ryzykownych (operacje na plikach, baza danych) użyj izolowanego kontenera:

```
docker run -it --rm \
  -v $(pwd):/workspace \
  -e ANTHROPIC_API_KEY=$ANTHROPIC_API_KEY \
  python:3.12-slim \
  bash -c "pip install aider-chat && cd /workspace && aider"
```

Kontener daje pełną izolację – przypadkowe zniszczenie plików ogranicza się do podmontowanego wolumenu, nie dotyczy reszty systemu.

CLI vs. IDE-based tools – praktyczne porównanie

Kryterium	Aider / Open Interpreter	Cursor / Copilot
Środowisko	Terminal, SSH, CI/CD, Docker	VS Code, JetBrains
Integracja z Git	Automatyczna (Aider)	Manualna
Wykonywanie kodu	✓ (Open Interpreter)	Ograniczone
Wizualizacja diff	Tekstowy diff	Graficzny diff
Modele lokalne	✓ (pełne wsparcie)	Wymaga połączenia

Rekomendowane przypadki użycia CLI: migracje danych, automatyzacja w CI/CD, środowiska serwerowe bez GUI, szybkie skrypty analityczne.

Aider w GitHub Actions – przykład integracji

```
# Automatyczna naprawa błędów lintingu w pipeline CI
- name: Fix lint errors with AI
  run: |
    pip install aider-chat
    aider --model gpt-4o-mini --no-auto-commits \
      --message "Napraw wszystkie błędy ESLint w plikach zmienionych w tym PR" \
      (gitdiff--name-onlyHEAD 1|grep'\.js')
```

Narzędzia CLI są szczególnie wartościowe w scenariuszach, gdzie edytor graficzny jest niedostępny lub zbędny – CI/CD, serwery zdalne, automatyzacja bez nadzoru człowieka.